National University of Computer and Emerging Sciences

CS3009 — Software Engineering | Spring 2026

Naawbi

Learning Management System

Deliverable 1

Requirements & Sprint 1 Documentation

Team Name	**Ibwaan**
Team Lead	**Ibraheem Farooq**
Members	Ibraheem Farooq, Wajih-Ur-Raza Asif, Ali Aan
Submission Date	28 February 2026
Repository	github.com/aliaankhowaja/Naawbi

ibbi1020 • wajih-rathore • aliaankhowaja

Contents

1 Introduction **3**
1.1 Context . . . 3
1.2 Problem Statement . . . 3
1.3 Goals . . . 3

2 Project Vision **3**

3 Project Team Information **3**
3.1 Team Roles . . . 4
3.2 Member Biographies . . . 4
3.3 Team Agreement . . . 4

4 Intended Use of the System **6**
4.1 Stakeholders . . . 6

5 Features and Overall Functionality **7**
5.1 What the System Will Do . . . 7
5.1.1 Authentication and Role-Based Access Control (RBAC) . . . 7
5.1.2 Course Enrollment and Management . . . 7
5.1.3 Announcements and Course Material . . . 7
5.1.4 Assignments and Submissions . . . 7
5.1.5 Grading, Feedback, and Gradebook . . . 8
5.1.6 To-Do List and Deadlines (Key Extension) . . . 8
5.2 How the System Helps Users . . . 8

6 User Stories **8**
6.1 Authentication and Role-Based Access Control (RBAC) . . . 8
6.2 Course Enrollment and Management . . . 9
6.3 Announcements and Course Material . . . 9
6.4 Assignments and Submissions . . . 9
6.5 Grading, Feedback, and Gradebook . . . 10
6.6 To-Do List and Deadlines (Key Extension) . . . 10

7 Structured Specifications for All User Stories **10**
7.1 Authentication and Role-Based Access Control (RBAC) . . . 10
7.2 Course Enrollment and Management . . . 11
7.3 Announcements and Course Material . . . 12
7.4 Assignments and Submissions . . . 12
7.5 Grading, Feedback, and Gradebook . . . 13

7.6 To-Do List and Deadlines (Key Extension) 14

8 Scrum Board **14**

8.1 Sprint 1 Module 14

8.2 Sprint 1 Selected User Stories 14

8.3 Four Structured Specifications (Sprint 1) 14

8.4 Scrum Board Snapshots (Trello) 15

8.5 Sprint 1 Product Burn-Down Chart 15

9 Non-Functional Requirements (NFR) Specification **16**

10 Iteration 1 Implementation Screenshots **16**

11 Work Division **17**

1 Introduction

1.1 Context

This project is a university Software Engineering team project to build a Learning Management System (LMS). The system will implement standard SE practices (requirements, design, implementation, testing, and documentation) and deliver a working product with an MVP scope plus selected extensions.

1.2 Problem Statement

Current course management for small-to-mid scale classes often suffers from fragmented tools for announcements, content, submissions, grading, and learner progress visibility. Students lack a clear, data-driven view of performance and deadlines. Instructors need a single workflow to publish content, collect submissions, grade with rubrics, and monitor class progress.

1.3 Goals

- Deliver a usable LMS that supports the end-to-end workflow for a course: **content** → **assignments** → **submissions** → **grading**.

2 Project Vision

Build a clean, reliable, course-centric LMS that instructors can use to manage learning materials and evaluations, and that students can use to track progress and performance. The key differentiators are:

- **Grading** — rubric-driven grading for assignments.
- **Analytics Dashboard** — performance, progress, and deadline dashboards for both students and faculty.

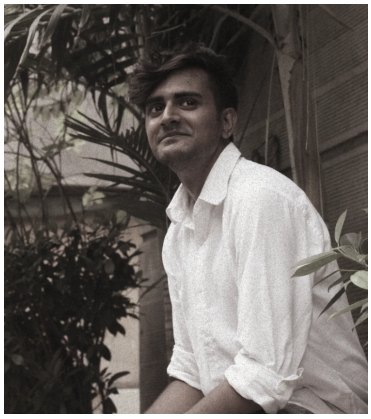
3 Project Team Information

Team Name	Ibwaan
Project Name	Naawbi
Team Lead	Ibraheem Farooq

3.1 Team Roles

Member	Role(s)	GitHub
Ibraheem Farooq	PM, Scrum Master	github.com/ibbi1020
Wajih-Ur-Raza Asif	Requirements Architect, Tester	github.com/wajih-rathore
Ali Aan	Developer, UI Designer	github.com/aliaankhowaja

3.2 Member Biographies



Ibraheem Farooq

PM / Scrum Master

A Computer Science major based in Islamabad who transitioned from designing posters and brand identities to UX and product design. His technical background provides a unique perspective on bridging digital experiences that balance user needs with technical viability. Passionate about solving real problems through thoughtful design and clean, intuitive interfaces.

Wajih-Ur-Raza Asif

Requirements Architect / Tester

A software engineer focused on building practical systems that solve real operational problems. His background spans end-to-end projects across different technologies, with a strong interest in automation, AI-driven workflows, and scalable system design. Currently expanding his understanding of enterprise technology and systems that make processes more efficient and reliable.

Ali Aan

Developer / UI Designer

An AI Engineer driven by a simple goal: building applications that work at scale. His expertise lies at the intersection of high-performance backend architecture and agentic systems. He specialises in designing and deploying scalable infrastructures that power intelligent applications. Beyond the code, he is a robotics enthusiast passionate about bringing new ideas to life.

3.3 Team Agreement

Communication

- **WhatsApp** — primary channel for daily coordination; respond within 1 hour.
- **In-person** — preferred when on campus.
- **Google Meet** — for remote meetings.
- Urgent issues: tag the team and respond within 20 minutes.
- If unavailable: post an async update in WhatsApp within 24 hours.

Meetings

- Mandatory: Sprint Planning, Sprint Review, Sprint Retrospective.
- Regular check-ins: 4× per week (in-person or Google Meet).
- Absence: notify team 24 hours in advance and share an async update.
- Every meeting ends with: decisions + action items + owners.
- Notes: rotating assigned person.

Meeting Preparation

- **Sprint Planning:** review backlog, propose task breakdowns.
- **Review:** prepare demo flow + screenshots.
- **Retrospective:** each member prepares (1) what went well, (2) what did not, (3) what to change.

Version Control

- Small, atomic commits; new branch per ticket.
- PR required before merging; do not commit secrets, credentials, or large binaries.
- Commit message format: `feat:`, `fix:`, `docs:`, `chore:`

Division of Work

Owner	Responsibility
Wajih	Requirement ownership: user stories, structured specs, and consistency checks.
Ali	UI ownership and feature implementation.
Ibraheem	Coordination, sprint tracking, packaging, and final submission.

Contingency Plan. If a member becomes unavailable or falls behind, the team will re-scope to the minimum required deliverable content, redistribute work immediately, and prioritise required artefacts. The PM/SM will update the board, assign new owners, and schedule an urgent meeting to unblock progress.

Submission Protocol

- **Submission owner:** Wajih.
- **Internal review:** Ibraheem and Ali.
- **Internal cutoff:** finalise 24 hours before the deadline.

4 Intended Use of the System

4.1 Stakeholders

Students

- View enrolled courses, announcements, and deadlines.
- Submit assignments.
- View grades and instructor comments.
- Use the analytics dashboard to understand:
  - current standing and grade breakdown;
  - missing/late work;
  - upcoming deadlines and workload.

Instructors

- Create and manage courses.
- Create assignments with due dates and submission rules.

- Grade submissions and provide feedback.
- View course-level insights (basic analytics):
  - submission rates;
  - grade distributions;
  - at-risk indicators.

5 Features and Overall Functionality

5.1 What the System Will Do

5.1.1 Authentication and Role-Based Access Control (RBAC)

- User self-registration with role selection (Student or Instructor).
- Login and logout.
- Roles: Instructor, Student (optional: Teaching Assistant).
- Access control enforced across all features.

5.1.2 Course Enrollment and Management

- Course creation (instructor).
- Course catalogue/listing.
- Enrollment workflow: students enroll using a randomly generated, unique code per class.
- Course home: stream of announcements and assignments, similar to Google Classroom.

5.1.3 Announcements and Course Material

- Instructors post announcements visible to enrolled students.
- Announcement feed per course.
- File-based course material upload and distribution.

5.1.4 Assignments and Submissions

Assignments are a sub-category of announcements with the added functionality of file upload and resubmission rules (similar to Google Classroom).

- Instructors create assignments with title, description, due date/time, and late policy.

- Students submit via file upload with configurable resubmission rules (e.g., allowed until due date).
- Threaded comments within assignments.
- Submission tracking: timestamped submissions with status (*submitted / late / missing*).

5.1.5 Grading, Feedback, and Gradebook

- Instructor grading interface: view submissions, assign scores, add comments.
- Gradebook:
  - Instructor view — all students and graded items.
  - Student view — personal grades and feedback.

5.1.6 To-Do List and Deadlines (Key Extension)

A simple list within the course catalogue section providing student-facing insights:

- **Completion/progress indicators:** submitted vs. missing items.
- **Deadline awareness:** upcoming due dates and late/missed items.

5.2 How the System Helps Users

- Consolidates course operations (content, announcements, assessments, grading) into one unified workflow.
- Reduces instructor workload through rubric reuse.
- Provides traceability for evaluation: all actions are recorded and auditable (who submitted what and when; who graded what and when).

6 User Stories

6.1 Authentication and Role-Based Access Control (RBAC)

US 1 As a *User* (Student/Instructor), I want to log in securely so that I can access my account and authorised system features.

US 2 As a *User*, I want to log out of my account so that my session is secure when I am not using the system.

US 3 As a *User*, I want to select my role (Student or Instructor) during registration so that I can use the platform appropriately.

US 4 As a *User*, I want role-based access control to be enforced across all features so that individuals can only perform actions permitted by their chosen role.

6.2 Course Enrollment and Management

US 5 As an *Instructor*, I want to create a new course so that I have a dedicated space to manage learning materials and track student progress.

US 6 As a *User*, I want to view a course catalogue/listing so that I can see the available courses in the system.

US 7 As an *Instructor*, I want the system to generate a unique random code for my course so that I can share it securely with my target students.

US 8 As a *Student*, I want to enroll in a course using a unique class code so that I can self-register for the classes I am supposed to take.

US 9 As a *Student*, I want to view a course home page that displays announcements and upcoming items so that I am immediately aware of current course activities.

6.3 Announcements and Course Material

US 10 As an *Instructor*, I want to post announcements so that I can broadcast important updates or instructions to all enrolled students.

US 11 As a *Student*, I want to see an announcement feed for my course so that I never miss critical information from my instructor.

US 12 As an *Instructor*, I want to upload and publish course materials so that my students have access to the necessary learning resources.

US 13 As a *Student*, I want to access and download course materials so that I can study and complete my assignments.

6.4 Assignments and Submissions

US 14 As an *Instructor*, I want to create assignments with a title, description, and due date/time so that expectations are clear to students.

US 15 As an *Instructor*, I want to define late policies and resubmission rules so that I can enforce grading standards.

US 16 As a *Student*, I want to submit my assignments via file upload so that I can turn in my completed work electronically.

US 17 As an *Instructor*, I want the system to timestamp all submissions so that I can accurately determine their status (submitted, late, missing).

US 18 As a *Student*, I want to see the status of my submissions (e.g., submitted on time, late, missing) so that I know my work has been received.

6.5 Grading, Feedback, and Gradebook

US 19 As an *Instructor*, I want to view a list of student submissions for an assignment so that I can review them efficiently.

US 20 As an *Instructor*, I want to assign scores and add comments to student submissions so that I can provide them with constructive feedback.

US 21 As an *Instructor*, I want to view a comprehensive gradebook of all students and their graded items so that I can monitor the overall performance of the class.

US 22 As a *Student*, I want to view my personal gradebook with scores and instructor feedback so that I can track my academic standing and progress.

6.6 To-Do List and Deadlines (Key Extension)

US 23 As a *Student*, I want to view a to-do list within my course catalogue section so that I have complete visibility over my progress (submitted vs. missing items).

US 24 As a *Student*, I want the system to highlight upcoming due dates and late/missed items so that I can effectively manage my deadlines and prioritise my workload.

7 Structured Specifications for All User Stories

7.1 Authentication and Role-Based Access Control (RBAC)

US 1 & US 2 — Secure Login and Logout

Roles: Student, Instructor

Description: Users must be able to authenticate themselves to access the system safely, and end their session when finished.

Acceptance Criteria:

- System presents a login form requiring email and password.
- System authenticates credentials against the database.
- Successful login redirects to the user's respective dashboard.
- System provides a visible logout button that terminates the session and redirects to the login page.

US 3 & US 4 — Registration and Role Enforcement

Roles: Student, Instructor

Description: Users can register an account, select their role, and the system consistently enforces access restrictions based on this role.

Acceptance Criteria:

- Registration page includes a required field to select "Student" or "Instructor" role.
- Students attempting to access instructor-only routes (e.g., course creation, grading) receive an "Access Denied" error.
- Instructors cannot access student-only views if not enrolled as a student.

7.2 Course Enrollment and Management

US 5 — Course Creation

Roles: Instructor

Description: Instructors can create courses to manage their classes.

Acceptance Criteria:

- Instructor can access a "Create Course" form.
- Form requires Course Name and optional description/syllabus.
- Upon creation, the course is added to the instructor's dashboard.

US 6 & US 9 — Course Catalogue and Home Page

Roles: Student, Instructor

Description: Users can view a list of their courses and access a centralised course home page featuring announcements and upcoming items.

Acceptance Criteria:

- Dashboard displays all active courses the user is enrolled in or teaching.
- Clicking a course opens the Course Home with a feed of recent announcements, latest assignments, and immediate deadlines (similar to Google Classroom's Stream).

US 7 & US 8 — Unique Class Code Enrollment

Roles: Student, Instructor

Description: The system generates a unique join code for courses, which students use to self-enroll.

Acceptance Criteria:

- Upon course creation, a unique alphanumeric code is automatically generated.
- Instructor's course settings page clearly displays the join code.
- Student dashboard has a "Join Course" button accepting a valid class code as

input.

- Entering a valid code adds the student to the course roster immediately.

7.3 Announcements and Course Material

US 10 & US 11 — Course Announcements

Roles: Student, Instructor

Description: Instructors publish text/media updates that populate a centralised feed for students.

Acceptance Criteria:

- Instructors have an input box on the Course Home to post an announcement.
- Announcements support basic text formatting.
- Upon posting, the announcement is instantly visible on the Course Home feed for all students in the course.

US 12 & US 13 — Course Material Distribution

Roles: Student, Instructor

Description: Instructors upload relevant class materials for student retrieval.

Acceptance Criteria:

- Instructors can attach files (PDFs, docs, links) to posts or a dedicated "Materials" section.
- Students can view or download the attached files without errors.

7.4 Assignments and Submissions

US 14 & US 15 — Assignment Creation and Policies

Roles: Instructor

Description: Instructors define tasks with explicit instructions, deadlines, and submission rules.

Acceptance Criteria:

- Instructor can create an assignment specifying Title, Description, Total Points, Due Date, and Due Time.
- Instructor can toggle late submission permissions (e.g., "Allow late turn-ins" vs. "Hard deadline").
- Assignment appears in the course feed and on students' To-Do lists once published.

US 16, US 17 & US 18 — Assignment Submissions and Tracking

Roles: Student, Instructor

Description: Students submit files, and the system records the exact turn-in timestamp, comment history, and completion status.

Acceptance Criteria:

- Students have a dedicated "Submit" area within each assignment to upload files.
- System records the exact server time of the upload.
- Submissions are automatically tagged as:
  - **Submitted** — if before deadline;
  - **Late** — if after deadline (provided late policy allows);
  - **Missing** — if deadline passed with no upload.
- Students can unsubmit/resubmit if the deadline has not passed and instructor settings allow it.
- Both students and instructors can leave threaded or inline comments on an assignment.

7.5 Grading, Feedback, and Gradebook

US 19 & US 20 — Instructor Grading Interface

Roles: Instructor

Description: Instructors evaluate student submissions, assign numeric scores, and leave qualitative comments.

Acceptance Criteria:

- Instructors can open a list of all enrolled students for a specific assignment.
- Instructors can quickly view or download associated submission files.
- Instructors can input a numeric grade (up to the maximum defined points) and a text comment.
- Grades and feedback are saved and immediately reflected for the respective student.

US 21 & US 22 — Comprehensive Gradebook Visibility

Roles: Student, Instructor

Description: Instructors see a macro-view of class performance, while students see their individual progress.

Acceptance Criteria:

- Instructor sees a grid/table where rows are students and columns are assignments,

with total grades calculated.

- Students access a personal "Grades" tab showing all their graded assignments, earned points, and instructor feedback in one list.

7.6 To-Do List and Deadlines (Key Extension)

US 23 & US 24 — Student Progress Tracking

Roles: Student

Description: Students receive automated insights into their workload, deadlines, and missing work.

Acceptance Criteria:

- Student dashboard features a "To-Do" section outlining all pending assignments across all enrolled courses.
- The "To-Do" list is sorted chronologically by approaching deadlines.
- Passed deadlines without submissions are prominently labelled as "Missing".
- Items are automatically removed from the active To-Do list once submitted by the student.

8 Scrum Board

8.1 Sprint 1 Module

Module Name	Course Management
Sprint Goal	Implement core course management features to get the ball rolling.

8.2 Sprint 1 Selected User Stories

[Placeholder — image not yet available]

Sprint 1 User Stories

Figure 1: Sprint 1 Selected User Stories (Trello screenshot)

8.3 Four Structured Specifications (Sprint 1)

The following structured specifications from Section 7 are selected for Sprint 1:

1. **US 1 & US 2:** Secure Login and Logout
2. **US 3 & US 4:** Registration and Role Enforcement
3. **US 5:** Course Creation

4. **US 7 & US 8:** Unique Class Code Enrollment

8.4 Scrum Board Snapshots (Trello)

[Placeholder — image not yet available]

Trello Start

Figure 2: Trello Board — Start of Sprint 1

[Placeholder — image not yet available]

Trello Mid

Figure 3: Trello Board — Mid-Sprint

[Placeholder — image not yet available]

Trello End

Figure 4: Trello Board — End of Sprint 1

8.5 Sprint 1 Product Burn-Down Chart

[Placeholder — image not yet available]

Burndown Chart

Figure 5: Sprint 1 Burn-Down Chart

9 Non-Functional Requirements (NFR) Specification

NFR Category	Requirement
Performance	Primary pages (course home, module list, gradebook summary) load within 2–3 seconds under normal class sizes (~200 students, a few courses) on typical lab hardware.
Security	RBAC enforced so students cannot access other students' private grades/submissions. User inputs are validated and sanitised to prevent common web vulnerabilities.
Reliability	Submissions are not lost once acknowledged. Errors are handled gracefully with actionable error messages.
Usability	Consistent UI across modules (common navigation and layout). Clear status indicators for submissions (*submitted / late / missing*).
Maintainability	Layered architecture (controller / service / repository). Unit tests for core business logic; at least basic integration tests for critical flows.
Scalability	Supports multiple courses and concurrent users typical of a department-scale deployment (not enterprise scale).
Auditability	Important events (logins, submissions, grading updates, enrollment changes) are logged for traceability.

10 Iteration 1 Implementation Screenshots

[Placeholder — image not yet available]

Iteration 1 Screenshot 1

Figure 6: Iteration 1 — Screenshot 1

[Placeholder — image not yet available]

Iteration 1 Screenshot 2

Figure 7: Iteration 1 — Screenshot 2

[Placeholder — image not yet available]

Iteration 1 Screenshot 3

Figure 8: Iteration 1 — Screenshot 3

11 Work Division

Ibraheem Farooq (PM / Scrum Master)

- Sprint planning, sprint tracking, and team coordination.
- Maintains delivery checklist for required artefacts and final submission packaging.
- Oversees branch/PR merge discipline and release readiness.

Wajih-Ur-Raza Asif (Requirements Architect)

- Requirements ownership: user stories, structured specifications, and consistency checks.
- Verifies Sprint 1 scope alignment with requirements and acceptance criteria.
- Supports validation of deliverable completeness.

Ali Aan (Developer / UI Designer)

- Development ownership for Sprint 1 scope (feature implementation).
- UI design ownership and implementation of user-facing screens.
- Produces Iteration 1 screenshots and demo-ready flows.